

Comments about AI

Marco Schrank

June 3, 2026

Contents

Introduction	1
Getting ollama up and running	2
Install ollama with homebrew	2
Run ollama as a service	2
Pull a model for local usage (hopefully open source dude)	2
Get your dev environment (VS Code) ready	2
Machine Learning	2
Neural Networks	3
Embeddings	3
Attention Mechanisms	3
Transformer Architektur	3
Reasoning	3
Agentic workflow	3
Agents	3

Introduction

This text is a curated not comprehensible mangle-mangle of topics mainly about LLMs (Large Language Models) based on the Transformer architecture. There is also an overview of ML Algorithm as they are still the best option for their designated purposes. If a ML Algorithm can do the job, don't use a deep neural network. Not every problem is a nail! Actually LLMs are plain math, some linear algebra and stochastics mainly. To get a deeper inside into the details of the Transformer architecture, it is necessary to hit the math behind. This text tries to explain the principals without math as far as possible. If you skip the math part, it should be possible to grab the meaning of the principals anyway. The harder (depends) part of each section is explicitly marked. As stated, at that point you should have got the meaning of the section. Some of the content is created or refined with AI!

Getting ollama up and running

Installation process is shown for the Mac with homebrew. This should not be much more difficult on other OS. It's assumed that you have homebrew on your Mac and you are able to install packages with it.

Install ollama with homebrew

```
brew install ollama
ollama --version
#get rid of it
brew uninstall ollama
```

Run ollama as a service

```
brew services start ollama
#you can also stop it that way
brew services stop ollama
```

If you don't want to run it as a service, just start it manually

```
ollama serve
#pull the model
#run the model
ollama pull mistral
```

Pull a model for local usage (hopefully open source dude)

Out of sovereignty, ecological and last but not least economical reasons, it's recommended to use a local model with at least free weights. Recommendation would be Mistral:latest because of the aforementioned reasons. As ollama supports the ChatGPT API, it is easy to change to a larger model in the cloud. The limiting factor for local LLMs is mostly the hardware at hand. Anyway, for most agentic scenarios and for demonstration purposes fits Mistral quit well. Let's pull the model:

```
ollama pull mistral
```

To see the list of models at hand:

```
ollama list
```

Get your dev environment (VS Code) ready

tbd...

```
{ "temperature": 0.7, "top_p": 0.9, "max_tokens": 2048 }
```

temperature: Kreativität (0.0 = deterministisch, 1.0 = zufällig). top_p: Diversität der Antworten (0.9 = gut für Code). max_tokens: Maximale Länge der Antwort.

Machine Learning

- kurzer überblick über gängige methoden
- abgrenzung zu neuronalen Netzen

Neural Networks

- aufbau/architektur
- inferenz
- training => backpropagation
- typen von neuronalen netzen

Embeddings

tbd... * was ist ein embedding? => fertige libs * was ist positional encoding?

Attention Mechanisms

tbd... * was ist attention? * wie wird attention trainiert?

Transformer Architektur

tbd... * Aufbau/Funktion Transformer Block erklärt * Warum mehrere Blöcke? Was folgt daraus? * Inferenz * Training von Transformer Blöcken und ganzen Transformern. Stichwort residuale Stapelung

Reasoning

tbd.. vielleicht

Agentic workflow

tbd...

Agents

tbd...

$$\alpha = \frac{\mathbf{K} \mathbf{Q}}{\sqrt{S}}$$

This is text with math: $E = m \cdot c^2$

convert to pdf with pandoc:

pandoc comments_about_ai.md -o comments_about_ai.pdf --pdf-engine=xelatex